



Exceptional service in the national interest

Simulation Tool for Asynchronous Cortical Streams (STACS)

Overview and Tutorial

Felix Wang

ICONS 2025 Tutorials

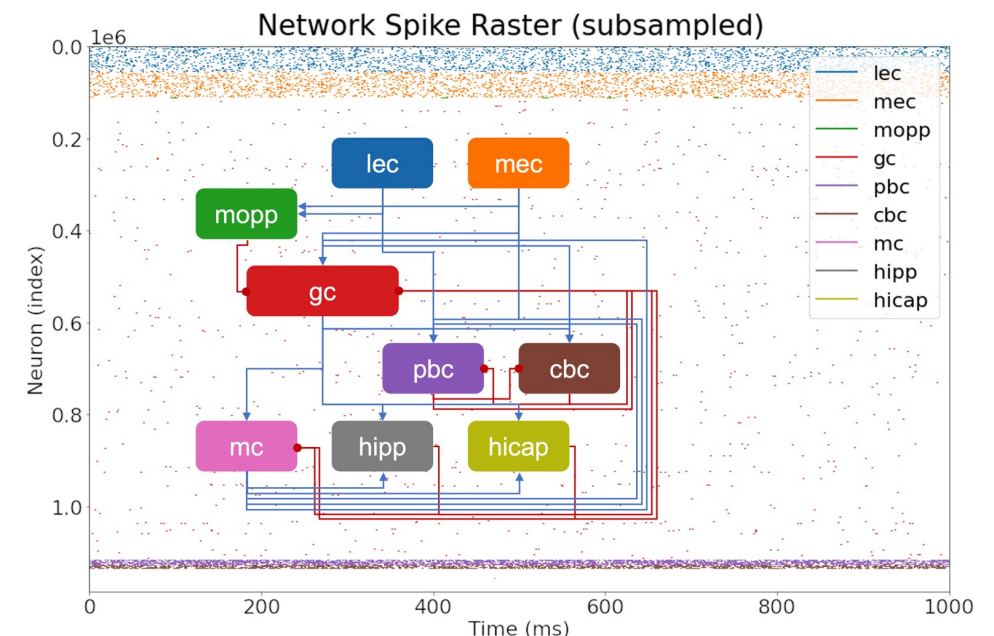


<https://github.com/sandialabs/STACS>

Simulation Tool for Asynchronous Cortical Streams

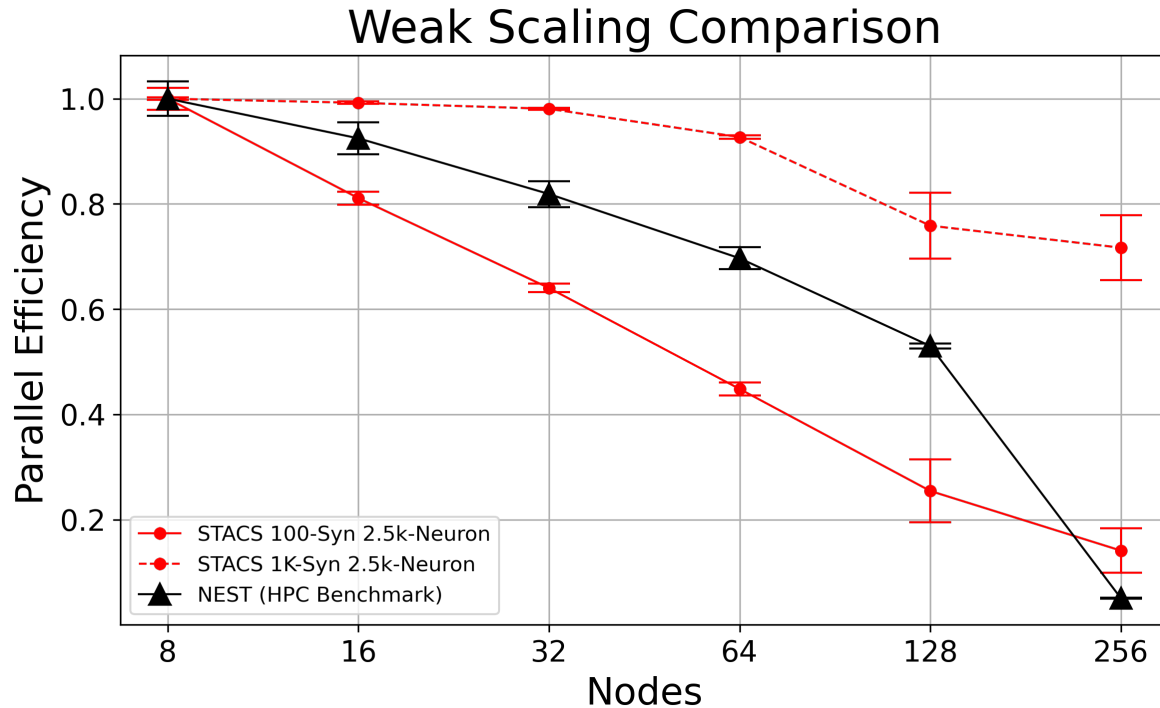
- STACS is a large-scale spiking neural network simulator built on top of the Charm++ parallel programming framework (runtime with parallel objects)
 - Network models are represented as a directed graph, and neuron (time-driven) and synapse (event-driven) dynamics are formulated as stochastic differential equations
 - Parallelization is supported by partition-based data structures, and serialized through SNN extensions to the distributed compressed sparse row (SNN-dCSR) data format
 - STACS provides scalable methods for graph generation, partitioning, simulation, serialization, and supports the neuromorphic computing toolchain as an interface between software (e.g. Fugu) and hardware (e.g. Loihi 2)
- Available at: <https://github.com/sandialabs/STACS>

Simulation of a biologically inspired spiking neural network of about 12M neurons and 70B synapses

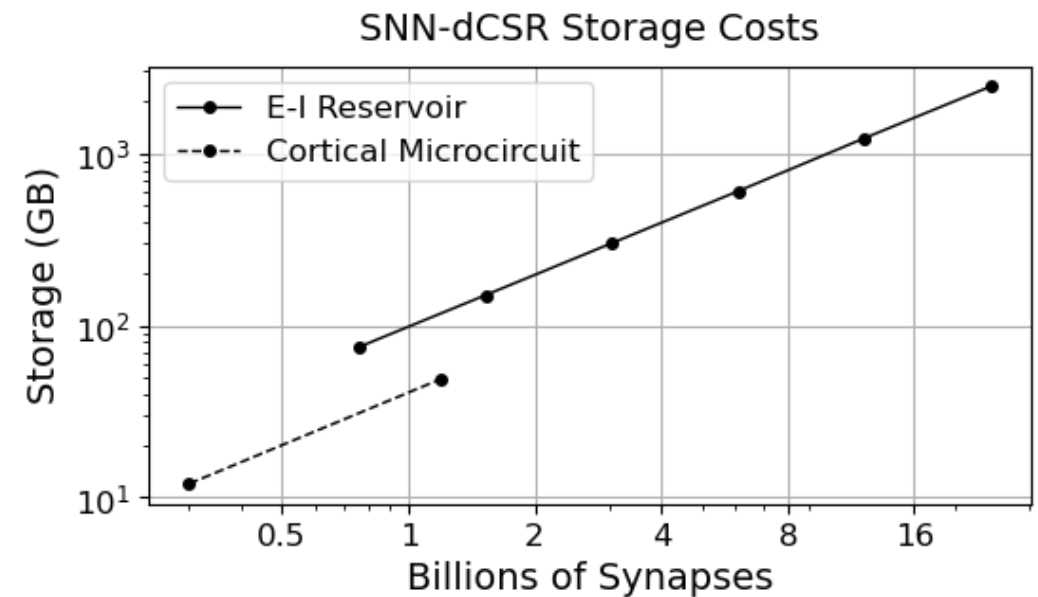


Selected features of STACS: Scalability

- Scaling experiments with a spatially parameterized E-I reservoir model showed $O(n)$ build times, vertex migration times, and storage costs, with competitive simulation time parallel efficiency at scale



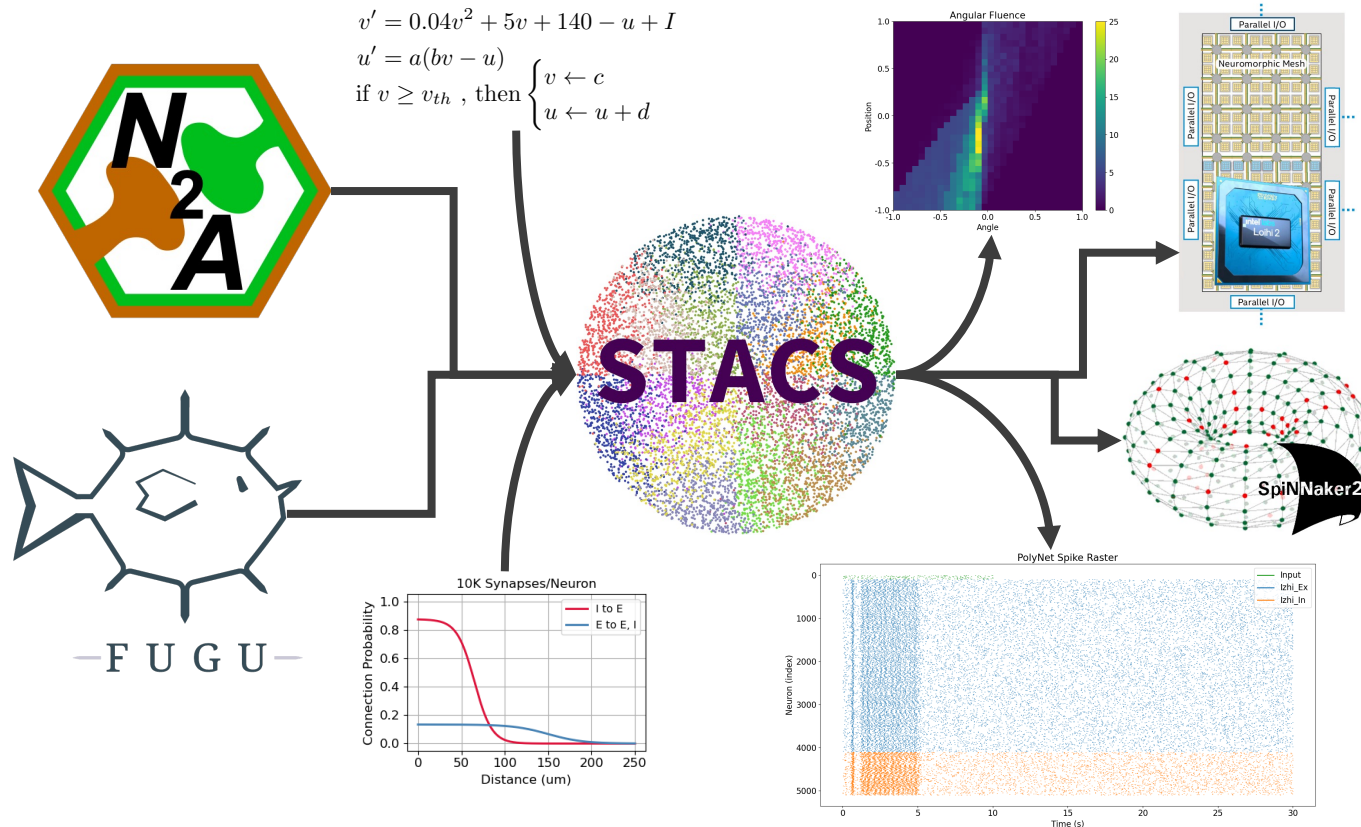
For this system, 256 nodes was 10240 CPU cores / partitions (for comparison, a Loihi 2 32-chip board has 3840 neurocores)



Storage costs are largely dominated by synaptic state, with minimal overhead from additional metadata files

Selected features of STACS: Interoperability

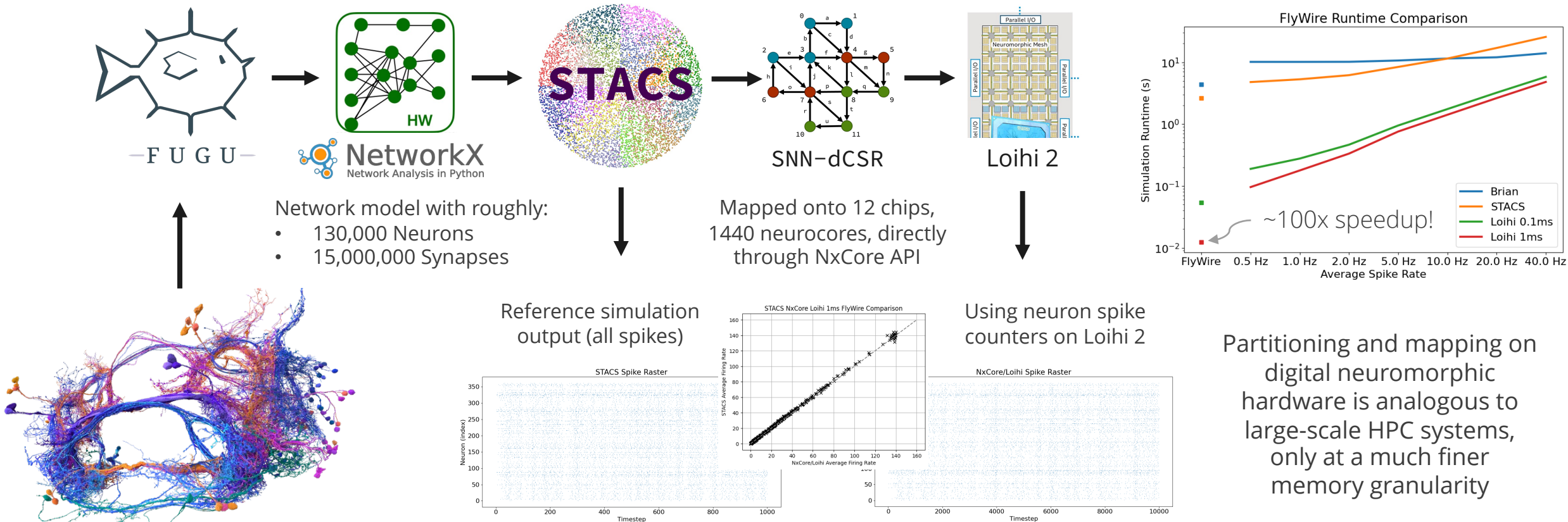
- STACS supports user-defined model dynamics which enables algorithm and model exploration or the emulation of hardware implementations



While STACS is primarily a spiking neural network simulator, it is interoperable with other tools and platforms through its intermediate representation for SNNs and portable text-based data formats (initially to work with graph partitioners such as ParMETIS)

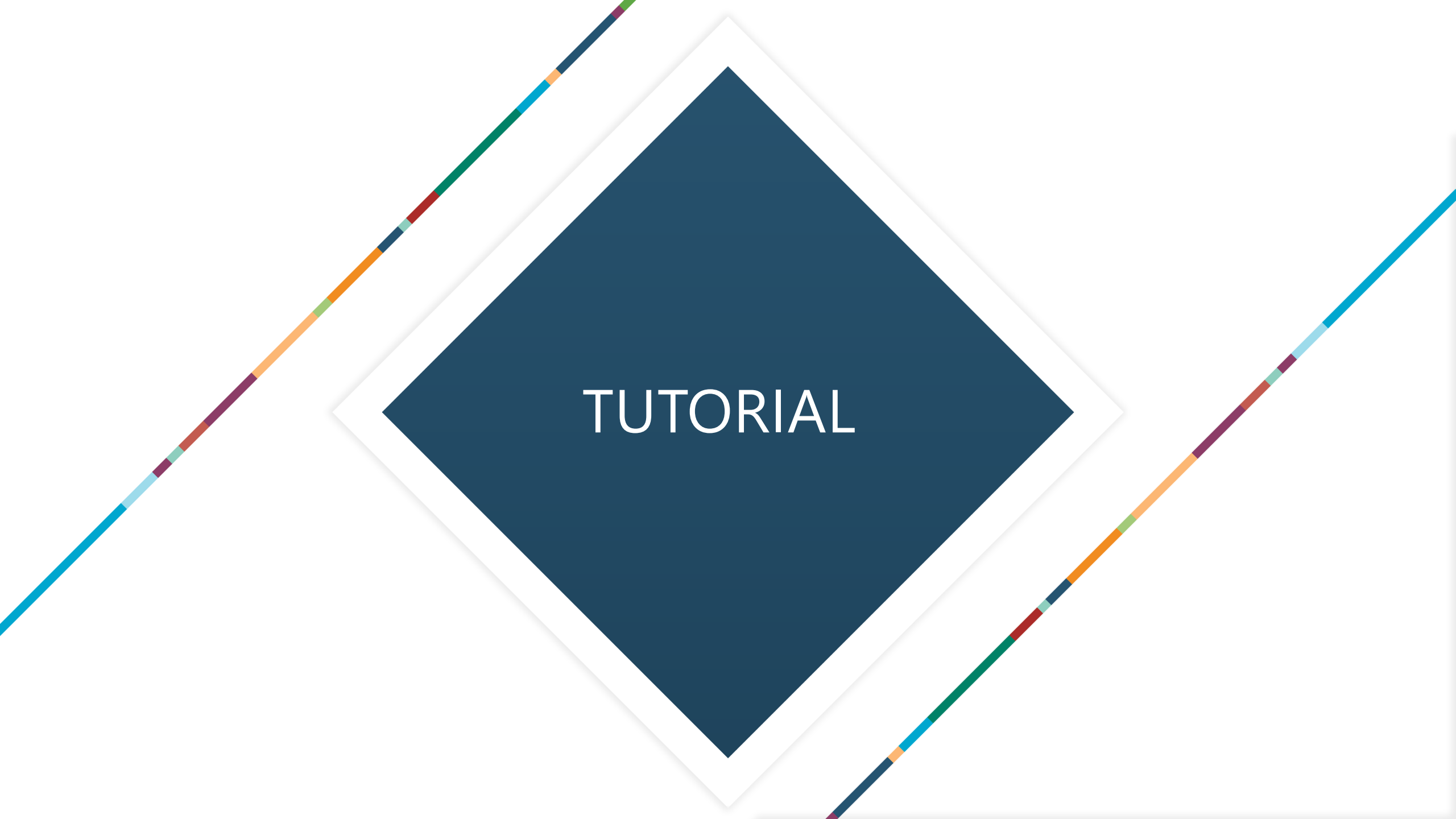
Robust compilation onto neuromorphic hardware

- We used the partition-based serialization in STACS as an intermediate representation to support mapping a network developed in Fugu and executed on the neurocores of Loihi 2



<https://codex.flywire.ai/>

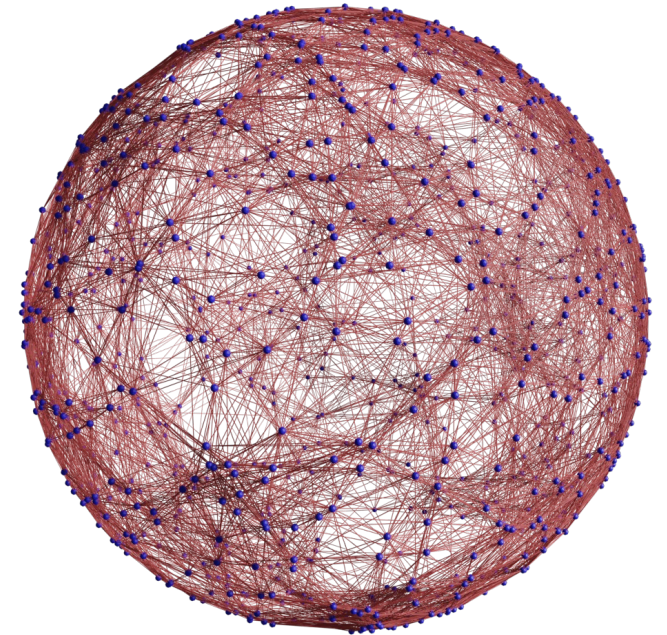
We validated our hardware implementation against expected spike rates

The graphic features a central dark blue diamond with a white border, containing the word 'TUTORIAL' in white. Two diagonal lines, each composed of several colored segments (cyan, light blue, purple, orange, green, red, dark blue), cross the diamond from the corners of the page.

TUTORIAL

Representing a Network Model in STACS

- Network models are represented as a directed graph, and neuron and synapse dynamics are formulated as stochastic differential equations
 - $G(V, E): V = \{v_1, v_2, \dots, v_n\} \quad E = \{e_1, e_2, \dots, e_m\}$
 where $e_k: v_i \rightarrow v_j \quad i, j \in 1, \dots, n \quad \text{and} \quad k \in 1, \dots, m$
 - $dx = f(x)dt + \sum_{i=1}^n g_i(x)dN_i$
 - Drift process** evolves state continuously in time
 - Diffuse processes** map to network events (e.g. spikes)



Visualization of a 6000 neuron network over the surface of a sphere

$$v' = 0.04v^2 + 5v + 140 - u + I$$

$$u' = a(bv - u)$$

if $v \geq v_{th}$, then $\begin{cases} v \leftarrow c \\ u \leftarrow u + d \end{cases}$

Izhikevich neuron equations (top)

$$\tau_+ \frac{dx_i}{dt} = -x_i + a_+(x_i) \sum_m \delta(t - t_i^m)$$

$$\tau_- \frac{dy_j}{dt} = -y_j + a_-(y_j) \sum_n \delta(t - t_j^n)$$

$$\frac{dw_{ij}}{dt} = A_{LTP}(w_{ij})x_i(t) \sum_n \delta(t - t_j^n)$$

$$- A_{LTD}(w_{ij})y_j(t) \sum_m \delta(t - t_i^m)$$

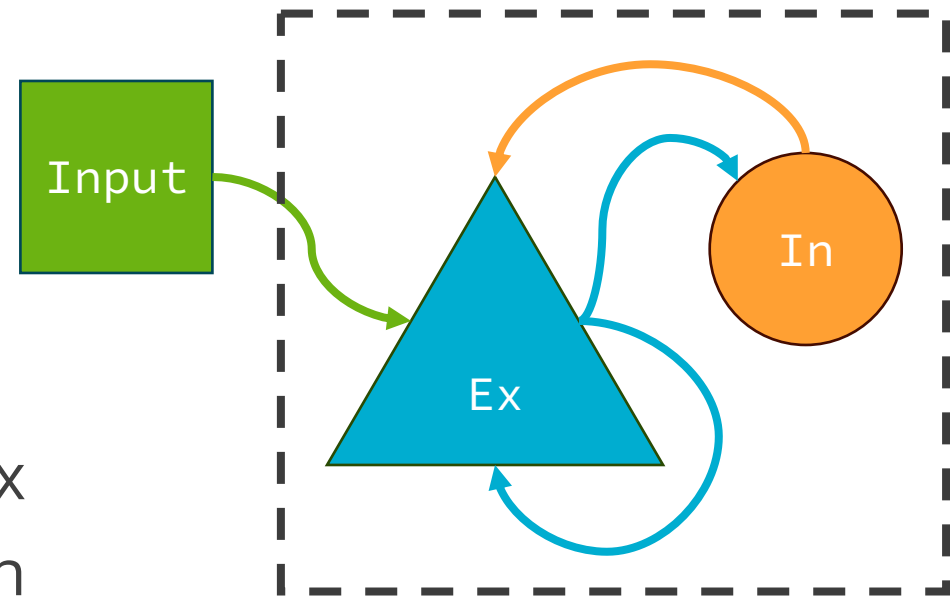
Trace-based STDP synaptic weight updates (right)

The structure of a STACS simulation experiment

- For a network, there's three YAML files that need to be defined
 - Network simulation config `.yaml` sets up information related to how the network will be processed by the simulator
 - Network graph model `.graph` defines the structure of the vertices and edges that compose the network
 - Network substrate model `.model` defines the substrate dynamics used by the vertices and edges
- These files are collected within a given directory (e.g. `./tutorial_net`)
 - Some useful sub-directories may be
 - `files` (for storing input or precomputed network structure/state)
 - `records` (for recording output)
- We'll be creating a polychronizing reservoir network for this tutorial
 - This is an 80/20 excitatory/inhibitory network with STDP (from Izhikevich 2006)

Simple Excitatory-Inhibitory Reservoir Network

- We will walk through the construction of a simple E-I network
- 4000 Excitatory Izhikevich neurons
- 1000 Inhibitory Izhikevich neurons
- 1 Hz background input to reservoir
- 100 user-provided input neurons
- Excitatory connections from Input $\rightarrow$ Ex
- Excitatory connections from Ex $\rightarrow$ Ex, In
- Inhibitory connections from In $\rightarrow$ Ex



Representing a Sparse Matrix (for the directed graph)

Sparse Matrix

	0	1	2	3	4	5	6	7	8	9
I	0				a			b		
	1		c			d				
	2						e		f	
	3			g				h		i
II	4	j			k		l		m	n
	5		o	p				q		
	6	r				s				
III	7			t		u				
	8					v				w
	9	x	y						z	

Compressed Sparse Row

Row: [0, 2, 4, 6, 9, 14, 17, 19, 21, 23, 26]

Col: [4, 7, 2, 5, 6, 8, 2, 7, 9, 0, 3, 5, 7, 9, 1, 2, 6, 0, 4, 3, 5, 5, 9, 0, 1, 8]

Val: [a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z]

Distributed Compressed Sparse Row

Part : [0, 4, 7, 10]

Row I : [0, 2, 4, 6, 9]

Row II : [0, 5, 8, 10]

Row III: [0, 2, 4, 7]

Col I : [4, 7, 2, 5, 6, 8, 2, 7, 9]

Col II : [0, 3, 5, 7, 9, 1, 2, 6, 0, 4]

Col III: [3, 5, 5, 9, 0, 1, 8]

Val I : [a, b, c, d, e, f, g, h, i]

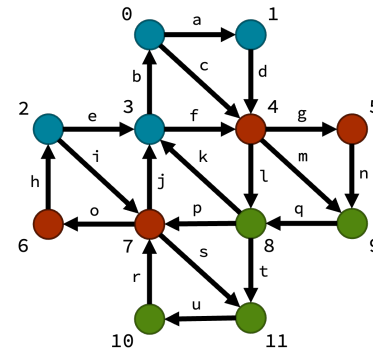
Val II : [j, k, l, m, n, o, p, q, r, s]

Val III: [t, u, v, w, x, y, z]

- Numbered values correspond to row and column coordinates
- Lettered values correspond to non-zero entries
- Roman numerals correspond to partitions
- Entries within a row are uniquely colored for ease of correlation within their corresponding arrays

Extending dCSR for SNNs

- The SNN extension to dCSR straightforwardly assigns neuron and synapse state tuples to the matrix values
- As a text-based file format, we may split the index and value information into separate, but aligned files
- Each partition provides “self-contained” dynamics except for external events



dist	adjcy.0	adjcy.1	adjcy.2
0 0	1 3 4	0 1 3 5 8 9	3 4 7 9 11
4 13	0 4	4 9	4 5 8
8 29	3 6 7	2 7	7 11
12 42	0 2 4 7 8	2 3 6 8 10 11	7 8 10
state.0	state.1	state.2	
0 $\emptyset$ b $\emptyset$	4 c d f $\emptyset$ $\emptyset$ $\emptyset$	8 $\emptyset$ l $\emptyset$ q $\emptyset$	
1 a $\emptyset$	5 g $\emptyset$	9 m n $\emptyset$	
2 $\emptyset$ h $\emptyset$	6 $\emptyset$ o	10 $\emptyset$ u	
3 $\emptyset$ e $\emptyset$ j k	7 i $\emptyset$ $\emptyset$ p r $\emptyset$	11 s t $\emptyset$	

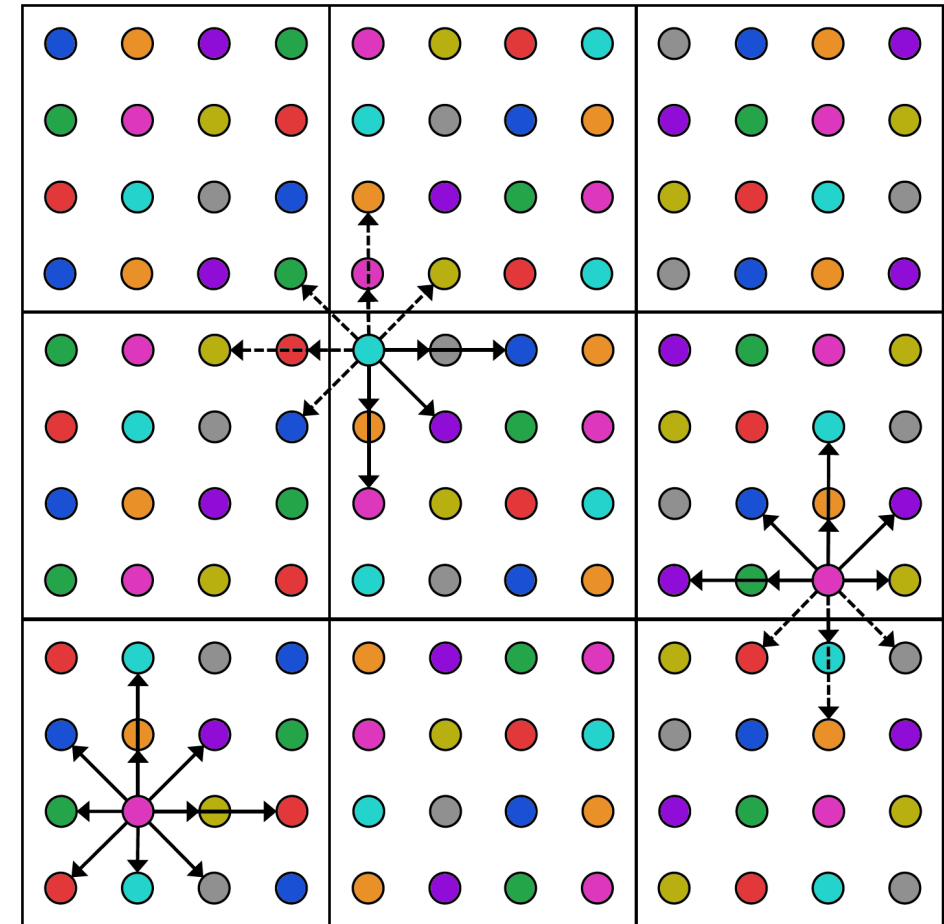
Example 3-way partitioning of a simple network with 12 vertices and 21 directed edges using the SNN-dCSR format. Directed edges with associated state are bolded.

- ‘Part’ information in `dist` file
- ‘Row’ information implicit in `adjcy` files (adjacency lists)
- ‘Col’ information explicit in `adjcy`
- ‘Val’ information in `state` files

Spatially Partitioning our Reservoir Network

- Partitioning improves memory locality and reduces communication costs
- The 'edge cut' (the number of edges that go between graph partitions) is a good metric for partitioning
- For a spatially distributed network, a naïve partitioning can work well

For this toy network, going from a 'round-robin' partitioning (by color) to a spatial partitioning (by grid) results in edge cut reduction from 1316 (88.2%) to 448 (32.7%)





BACKUP/EXTRA

Installation (Environment)

- The hardest part of installing STACS is setting up the Charm++ environment
 - Charm++ is actually quite portable, and can run on basically any machine
- Setting up the dependencies/environment (Linux, MacOS)
 - General development environment
 - `sudo apt-get install build-essential gfortran cmake`
 - Install Homebrew: <https://brew.sh/>
 - `brew install wget htop gcc cmake git`
 - STACS specific libraries
 - `sudo apt-get install libyaml-cpp-dev libfftw3-dev`
 - `brew install yaml-cpp fftw`
 - MPI (communication method)
 - `sudo apt-get install mpich`
 - `brew install mpich`

Installation (Charm++)

- Setting up Charm++ (Linux, MacOS, both)
 - Clone repo from github: <https://github.com/UIUC-PPL/charm>
 - Alternatively, get version 8.0.0 (available as tarball, zip)
 - `./build charm++ mpi-linux-x86_64 -j8`
 - If on Intel Mac, set the min version variable in the file `charm/src/arch/mpi-darwin-x86_64` to 12.6
 - `./build charm++ mpi-darwin-arm8 -j8`
 - Set up associated environment variables (e.g. in `.bashrc`)
 - `export PATH=/path/to/charm/bin:$PATH`
- There are quite a few additional settings available for Charm++
 - On a different OS (e.g. Windows, IBM Blue Gene/Q, Cray XE/XC, PowerPC)
 - Using a different communication substrate than MPI (e.g. IB verbs, multicore)
 - Using a different compiler (e.g. `icc`, `xlc`)
 - More information in `charm/README.md`

Installation (STACS)

- After Charm++ is installed, STACS uses a trivial Makefile to compile
- Setting up STACS
 - Clone repo from github: <https://github.com/sandialabs/STACS>
 - `make -j8`
- Setting up Python
 - Use your typical python environment (e.g. Anaconda)
 - `pip install pyyaml`
- STACS is a command line executable, which is great for running large batch jobs on an HPC allocation, but the workflow(s) can be unfamiliar to those mainly used to single-node development and working within Python
 - STACS is also just not as user-friendly as it could be (work in progress)
 - There are some ways we can loosely interface STACS with Python
 - Developing YAML configuration files through the pyyaml package
 - Parsing and importing simulation output (e.g. to display a spike raster)

Defining a Network Model

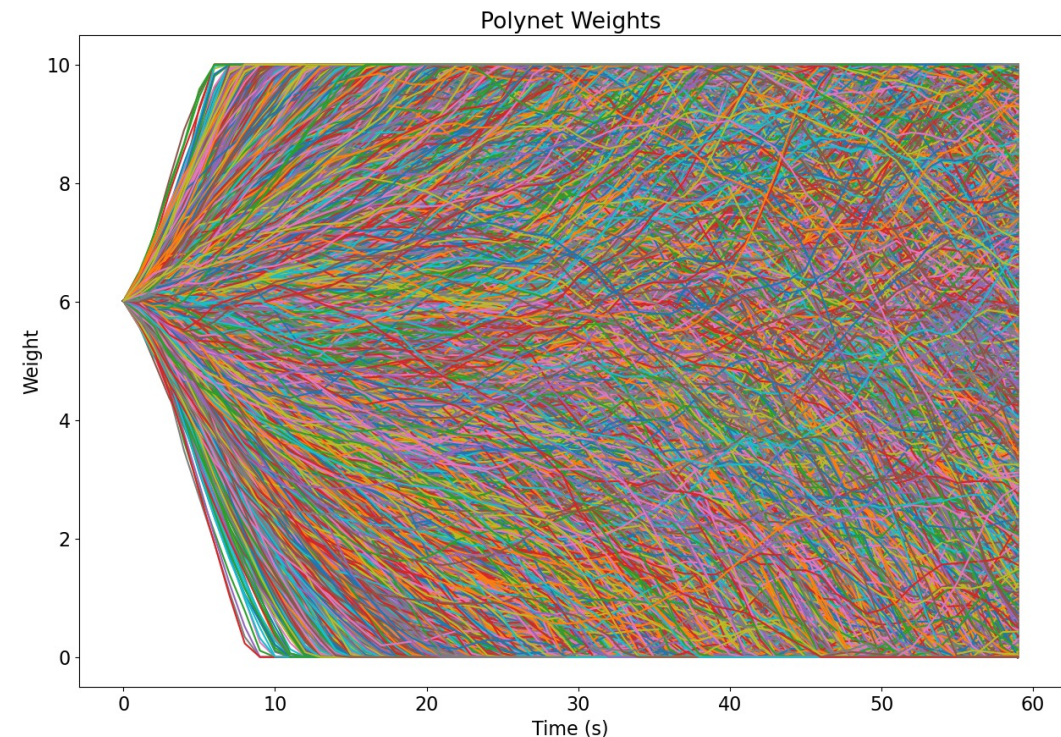
- A network model is composed of three YAML-formatted configuration files:
 - The main config file sets up information related to how the network model will be processed by the simulator (this was `polynet/polynet.yml`)
 - Parallelization, total simulation time, random number seed, episodic trials
 - The network graph file defines the connectivity structure of the vertices and edges that compose the network (this was `polynet/polynet.graph`)
 - Neuron populations and synaptic projections
 - The network substrate model file parameterizes the time- and event-driven dynamics used by the vertices and edges (this was `polynet/polynet.model`)
 - State initialization (potentially from file) during network generation
 - Setting up event watchers and state probes
 - Parameterizing streams which open up ports for I/O

Defining a Network Model (Vertices and Edges)

- Separated into multiple YAML “files” using the using the ``---`` and ``...`` syntax
- Vertices and edges both mostly follow the same model format
- Model names (`modname`) are user-defined strings and are used to differentiate between different parameterizations of substrate dynamics (model names should be unique)
- Model types (`modtype`, the integer identifier) are defined in the model files (located in `stacs/models`) and provide a pointer to the substrate dynamics being used
- Parameters (`param`): information that is related to the model dynamics, but are otherwise shared across all instantiations of a given model type (e.g. the membrane leakage rate for a given neuron population)
 - These are set as simple name-value pairs
- States (`state`): information that are unique per model instantiation and/or are mutable throughout a network simulation (e.g. the membrane voltage for a given neuron)
 - These are set as initialization parameters for network generation (e.g. random number drawn from a uniform interval, bounded normal distribution, etc.)

Defining a Network Model (Streams, Records)

- In addition to vertices and edges, another substrate type is a stream
 - These were originally used to connect a network to external devices through YARP
 - We can also use these to provide inputs from file (e.g. a spike generator)
 - For a simple inputs, the SpikeInput model reads in a YAML-formatted event list
 - These are parameterized as the number of inputs and the path to the file
- For recording more information from a network model, a record “file” can be used
 - Different event types (applied current stimulus, “clamped” spikes)
 - State probes (defined by the model name, state, and recording frequency)
 - These work for both vertices and edges (indexed by vertex, local edge)



Defining a Network Model (Graph)

- There are two main sections in the network graph file, a list of vertices and a list of edges
 - There is also a section for streams for communication in/out of the network
 - The `modname` references the parameterized substrate models from earlier
- Vertices define groups/populations, and edges define how these are connected
 - The `order` of a vertex entry is how many there are in the group
 - Edge entries have a `source` group and a list of `target` groups
 - Vertex model names and source-target pairs have to be unique
- Vertices allow for spatial instantiation parameters (e.g. within a circle, rectangle)
- Edges are instantiated based on information from the source-target vertices
 - Spatial connectivity parameters (e.g. cutoff distance, periodic boundaries)
 - Indexical connectivity parameters (e.g. local group index, offsets)
 - Connectivity matrix can also be supplied from file (target-major order)
- This system allows for quite a bit of freedom in mixing/matching different substrate models with each other (however, this also includes incompatible models)

Defining a Network Model (Simulation)

- The simulation parameters are what you would typically expect out of SNN simulators
 - Main simulation information
 - Different run modes (network generation, repartitioning, and simulation)
 - These can be overridden by passing a runmode to STACS
 - Random number seeds, plasticity, load balancing
 - Network storage information
 - Location to save snapshots and records, filenames
 - Number of files and network partitions to use
 - Timing information
 - Maximum simulation time, timestep, length of event queue (max delay)
 - How often to save records, checkpoint network
 - Episodic simulation number of trials and duration
- Constructing configuration files is mostly fill-in-the-blank

Building and Running a Network Model

- To run a network, we point STACS to the simulation config
 - `./charmrun +p8 ./stacs ./networks/polynet/polynet.yml buildsim`

$\underbrace{\quad\quad\quad}_{\text{charm++ launcher}}$
 $\underbrace{\quad\quad}_{\text{num processes to use}}$

$\underbrace{\quad\quad}_{\text{stacs executable}}$

$\underbrace{\quad\quad\quad\quad\quad}_{\text{network simulation config file}}$

$\underbrace{\quad\quad}_{\text{runmode}}$
- The run command is composed of a few different components
 - Charm++ launcher (this should also just be on the \$PATH)
 - Number of Charm++ processes to use (e.g. number of network partitions)
 - Optional path to an MPI hostfile (`[-f mpi_hostfile]`, if running on a cluster)
 - Path to the STACS executable (relative to current directory)
 - Path to the network simulation configuration file (relative to current directory)
 - Optional runmode (`buildsim` (`build`, `simulate`), `migrate` / `repart`)
 - Migrate/repart methods require `.part` files that provide new partition indexes (produced from a network partitioner, e.g. ParMETIS)

Building and Running a Network Model

- An example workflow for running an experiment could be:
 - Construct a network model
 - `network.yml`, `network.graph`, `network.model`
 - Build the network
 - `./charmrun +pN ./stacs /path/to/network.yml build`
 - Feed into a partitioner
 - `./charmrun +pN ./genet /path/to/network.yml part`
 - (GeNet is an earlier application that connects SNN-dCSR to ParMETIS)
 - Migrate the network vertices (overwrites original network snapshot)
 - `./charmrun +pN ./stacs /path/to/network.yml migrate`
 - Simulate the network for some time
 - `./charmrun +pN ./stacs /path/to/network.yml simulate`
 - Analyze the output (e.g. in Python, Matlab, etc.)
 - Network data in `.adjcy`, `.state`, `record/network.evtlog`, etc.

Building and Running a Network Model

- Some other ways to use STACS:
 - Build the network and simulate it
 - `./charmrun +pN ./stacs /path/to/network.yml buildsim`
 - Copy over and overwrite the original simulation output
 - ```
for i in $(ls network.out.*)
do j="$(echo ${i} | sed -e 's/.out//')"
```

```
mv "${i}" "${j}"
```

```
done
```
  - Modify the model parameterization (e.g. turn plasticity off, change out dynamics)
    - In `network.yml`, set `plastic = no`
    - In `network.model`, change `modtype = 20` to `modtype = 24`
      - When changing model types, the states must actually match between them
  - Simulate the network some more
    - `./charmrun +pN ./stacs /path/to/network.yml simulate`

# Developing Network Substrate Models

- All the STACS model files are found in `stacs/models`, and are defined as derived classes from the `NetModel` base class (using the `ModelTpl` template, factory method)
  - Network models are represented as a directed graph, and neuron and synapse model dynamics are formulated as stochastic differential equations:
    - $dx = f(x)dt + \sum_{i=1}^n g_i(x)dN_i$  : time-driven computation, event-driven communication
- When defining a network model, an integer identifier (the `modtype`) must be supplied
- Within the class constructure, there are multiple named lists for model parameters, model state (for real-valued and "tick" representations), auxiliary states (for allowing an edge model to access and modify vertex state, e.g. synaptic current), and ports
- For the neurons (`vertices`), the dynamics are located in the `Step` function, which are executed at each timestep
- For the synapses (`edges`), the dynamics are located in the `Jump` function, which are executed at discrete event times
- Other functions that may be `Reset` (for episodic trials) and `Leap` (for periodic updates)



# Developing Vertex Models (Step)

- For neuron models, we define how the states change over time
  - Model equations/dynamics are a function of model parameters and states
  - There is a lot of freedom available in how dynamics are computed
    - There are some random number generators available (uniform, normal)
    - Different numerical integration methods can be used (Euler, Runge-Kutta, etc.)
- Time is provided as the current simulation time `tdrift` and the amount of time left to simulate in the cycle `tdiff`
  - The “tick” representation in STACS is an integer type for precision
  - This can be converted to a real number (float type) through `TICKS_PER_MS`
- Any events (e.g. spikes) that are generated by the model are added to an event list
  - Events are defined as a `diffuse` timestamp, event `type`, and optional payload `data`
  - Helper indexical variables (`source` and `index`) are used for different target rules
    - This is any combination of local (self) and remote (target) vertices and edges

# Developing Edge Models (Jump)

- For edge models, we define how the states change with discrete event
  - Model equations/dynamics are now also a function of event information, and auxiliary states (corresponding to named states on the target vertex model)
  - There is still a lot of freedom available in how dynamics are computed
- All edge models must have delay set as its first “tick” type state variable
  - This is used to place events into their correct event queues
  - The delay must also be long as the timestep resolution used
- Although STACS mostly follows a point-neuron model, the vertex and (incoming) edge states are collated, which enables potentially interleaved dynamics (e.g. dendrites)
  - The list of vertex states are on index `[0]`, and the `auxstate` provides an indexical pointer to the relevant state in this list
- Information about where the event came from is provided in the `event.source` variable and may be used to select between different state updates (e.g. either side of STDP)
- If time since last spike is required, it will need to be saved in an edge state in the model